

Guide développeur — Atelier / TP C

Guide d'orientation pour un développeur qui reprend le code. Objectif : se repérer vite, savoir compiler/tester/empaqueter, et étendre le contenu sans casser le reste. Document **interne au dépôt** (non livré dans le bundle).

Références plus profondes, à préférer à toute paraphrase ici :

- `SPEC-plateforme-tp-snake.md` , `PLAN-plateforme-tp-snake.md` — spécification et plan d'origine de la plateforme.
- `RECONSTRUCTION.md` — reconstruire l'exe/bundle et publier une Release GitHub.
- `DOC-gestion-niveaux.md` — liste complète des champs de `meta.json` et du modèle de contenu.
- `HANDOFF-audit-2026-07-21.md` — état de livraison et branches.
- `GUIDE.md` — guide **utilisateur** (celui qui part dans le bundle sous le nom `README.md`).

1. Vue d'ensemble & structure du dépôt

L'atelier est une application de bureau **PyQt6** : 14 exercices d'introduction au C (parcours `be_c`), avec compilation gcc et correction automatique intégrées, plus un tuteur IA optionnel bridé. Il est distribué sous forme de **bundle Windows portable** construit avec PyInstaller.

Ce dépôt contient **le code source seulement**. Le bundle lançable (Python figé + gcc, ~300 Mo après élagage) n'est **pas versionné** : voir `README.md` . Cloner ne suffit pas pour lancer ; on lance soit depuis un Python normal (dév), soit depuis le bundle (élève).

Arborescence de premier niveau :

Chemin	Rôle
<code>*.py</code> (racine)	Modules de l'application (à plat, pas de package).
<code>contenu/<nom>/</code>	Parcours pédagogiques (données). <code>be_c</code> = les 14 exercices livrés.
<code>packaging/</code>	Point d'entrée du bundle (<code>entree_be_c.py</code>), scripts de build, lanceurs <code>.bat</code> / <code>.sh</code> .
<code>outils/</code>	Bancs de test (correcteur, tuteur) et pipeline de doc (captures, PDF).
<code>tests/</code>	Tests unitaires (<code>unittest</code>).
<code>w64devkit/</code> , <code>clangd/</code>	gcc (MinGW) et clangd, git-ignorés , posés à la racine par le build.
<code>.build/</code> , <code>_bundle/</code>	Sorties de build, git-ignorées.
<code>projet-corrige/</code> , <code>projet-squelette/</code> , <code>compagnon/</code> , <code>docs/</code> , <code>captures/</code>	Contenu projet SNAKE, compagnon Moodle, doc, captures.
<code>NOTE-*</code> , <code>PLAN-*</code> , <code>SPEC-*</code> , <code>REPONSE-*</code> , <code>BRIEFING-*</code>	Nombreux documents de conception/échange (contexte historique).

`progression.json` , `reglages.json` , `auteur.json` , `moodle_sync.json` , `releve.txt` et `journaux/` sont des **artefacts locaux** écrits à l'exécution, git-ignorés.

2. Carte des modules

Rôles dérivés des docstrings, vérifiés par lecture. Le dépôt est **plat** : tout module racine s'importe par son nom (`import chemins`). Aucune logique métier dans l'UI, aucune UI dans les modules d'exécution/notation.

Interface (PyQt6)

Module	Rôle
<code>fenetre.py</code>	Fenêtre principale. Câble énoncé, éditeur, console, tuteur et portes. Moteur commun aux parcours isolés et projet ; aiguille sur <code>etape.mode</code> (voir §3).
<code>dialogue_niveaux.py</code>	Boîtes de dialogue du mode auteur : gérer/réordonner les niveaux d'un parcours.
<code>dialogue_diagnostic.py</code>	Fenêtre « Emplacements et diagnostic », équivalent graphique de <code>diagnostic.bat</code> .
<code>theme.py</code>	Thème sombre (palette Fusion + feuille de style Qt).
<code>coloration.py</code>	Coloration syntaxique du C dans l'éditeur, purement lexicale.

Exécution & notation

Module	Rôle
<code>executeur.py</code>	Compile/exécute du C, rend un <code>Resultat</code> . Toutes les « portes ». Le code de sortie fait foi , pas le texte. Aucune UI.
<code>modele_etape.py</code>	Charge les étapes/parcours depuis les données (<code>meta.json</code> , <code>parcours.json</code>). Dataclasses <code>Etape</code> / <code>Parcours</code> . Aucune compilation.
<code>progression.py</code>	État persistant de l'étudiant (<code>progression.json</code>) et règles de déverrouillage.

Tuteur IA

Module	Rôle
<code>tuteur_ia.py</code>	Construit le prompt selon le cran, appelle le moteur en sous-processus, filtre lexical anti-solution. Résolution du moteur (voir §4).
<code>garde_fous.py</code>	Garde-fou structurel : recompile le code proposé par l'IA et rejoue la porte ; masque s'il ouvrirait vraiment la porte. Complète le filtre lexical.

LSP

Module	Rôle
<code>lsp_clangd.py</code>	Diagnostics en direct via clangd (diagnostics uniquement , pas d'autocomplétion — choix pédagogique). Se tait proprement si clangd absent.

Rédaction de contenu (outils auteur)

Module	Rôle
<code>atelier_contenu.py</code>	Crée/vérifie/retire parcours et étapes sans toucher au code. Bibliothèque standard uniquement.
<code>gestion_niveaux.py</code>	Ajouter, retirer, réordonner les niveaux d'un parcours (logique derrière <code>dialogue_niveaux</code>).

Plateforme / configuration

Module	Rôle
<code>chemins.py</code>	Chemins et drapeaux de compilation. <code>RACINE</code> , <code>CONTENU</code> , <code>SANS_FENETRE</code> , alignement clangd sur gcc, helpers pkg-config SDL. Aucune logique métier.
<code>reglages.py</code>	Réglages locaux mémorisés (<code>reglages.json</code>) : dernier parcours, tuteur actif, commande IA, <code>tout_debloque</code> . Sans PyQt.
<code>auteur.py</code>	Mode auteur : hachage/vérification du mot de passe protégeant l'édition de contenu.
<code>diagnostic.py</code>	Diagnostic système : où sont les choses, quels outils sont présents.

Moodle (optionnel)

Module	Rôle
<code>moodle_sync.py</code>	Pont optionnel vers le compagnon Moodle (LTI). Inerte sans appairage. Bibliothèque standard seulement.

Divers

Module	Rôle
<code>atelier_snake.py</code>	Point d'entrée de l'appli et des autotests (<code>--selftest</code> , <code>--releve</code> , <code>--smoketest</code> , <code>--demo</code> , <code>--parcours</code>).
<code>journal_session.py</code>	Journal d'événements de session (<code>journals/</code>), pour l'analyse pédagogique.
<code>releve.py</code>	Relevé de progression lisible sans écran ni serveur (mode local).
<code>espace_projet.py</code>	Gère la copie de travail du parcours projet SNAKE. Aucune compilation.

Note de nommage : la dataclass `Etape` déclare `type` $\in$ `{"perso", "jalon", "projet"}` en commentaire, mais les étapes `be_c` portent `"type": "programme"` . Le champ `type` n'est **pas** validé et n'est pas ce qui aiguille l'exécution : c'est `etape.mode` qui décide de la porte (voir §3). Le commentaire est donc indicatif, pas normatif.

3. Modèle de contenu

Un **parcours** = un dossier `contenu/<nom>/` avec un `parcours.json` :

- `ordre` : liste ordonnée des dossiers d'étapes.
- `mode` : "isole" (défaut) ou "projet".
- `libre` (optionnel) : `true` ouvre toutes les étapes d'emblée (révision), sans passer les portes précédentes. C'est un champ du **parcours**, pas un test sur son nom.

Une **étape** = un dossier avec `meta.json` + les fichiers pédagogiques (`enonce.md`, `starter.c`, `corrige.c`, et selon le mode `approfondissement.md`, `tests.c`, `corrige_buggue.c`, `stubs.c`, `test_reference.c`, `aperçu.c` ...).

Modes d'étape (champ `meta.mode`) et porte associée, aiguillage dans `fenetre.py` (bouton « Tester ») vers `executeur.py` :

mode	Porte appelée	Ce qui est jugé
programme	porte_programme	L'étudiant écrit un programme complet (son <code>main</code>). Exécuté avec <code>entree</code> , la sortie doit contenir les <code>sortie_attendue</code> (littéraux) et satisfaire les <code>sortie_motifs</code> (regex). Portes multi- cas possibles.
test_fourni	porte_perso	Le code de l'étudiant est compilé avec le <code>tests.c</code> fourni ; code de sortie 0 = porte ouverte.
test_a_ecrire	juger_test puis porte_jalon	L'étudiant écrit le test : il doit passer le corrigé et attraper <code>corrige_buggue.c</code> , puis son test valide son propre code.
(parcours projet)	porte_logique / construire_et_jouer_projet	Harnais logiques SDL et build final du jeu SNAKE.

Mécanique des « portes » (toutes dans `executeur.py`, préfixe `porte_*`) :

- Le **code de sortie du binaire fait foi** ; le texte n'est qu'affiché.
- `sortie_attendue` = fragments littéraux exigés ; `sortie_motifs` = {"motif": regex, "attendu": libellé}.
- `fragment_present()` évite qu'un fragment « = 5 » soit validé par une sortie « = 50 » (frontière numérique), tout en tolérant 9.9 vs 9.900000.
- **Portes étanches** (cas) : plusieurs jeux d'entrées, **tous** doivent passer. Un seul jeu laisserait passer un programme qui réimprime la sortie attendue en dur. Un cas hérite des `sortie_motifs` (format) mais **pas** des `sortie_attendue` (valeur).
- Sous-processus plafonné en mémoire et en temps (`_executer_cape`, cap 10 Mo, timeout 15 s).

Pour la **liste exhaustive des champs de meta.json**, se référer à `DOC-gestion-niveaux.md` (non dupliqué ici).

Parcours présents dans `contenu/` : `be_c` (14 exos, livré), `tp_c`, `perso`, `hybride` (défaut de `chemins.CONTENU`), `bonus_pointeurs`, `projet` (SNAKE, mode `projet`).

4. Chemins d'exécution & pièges (tous vérifiés)

- **Racine et fichiers d'état** — `chemins.RACINE = Path(__file__).resolve().parent`. `PROGRESSION_FICHER`, `REGLAGES_FICHER`, `AUTEUR_FICHER`, `MOODLE_SYNC_FICHER` sont sous `RACINE`. Dans l'**exe figé**, les modules vivent dans `_internal\`, donc ces JSON s'écrivent sous `_internal\` (le build nettoie ces résidus aux deux emplacements — voir `build_windows.ps1`). Helper : `chemins.contenu_racine(nom)`. `chemins.CONTENU` pointe sur `contenu/hybride` (héritage ; le parcours réel est choisi au lancement, pas via cette constante).
- **Piège d'encodage Windows** — tout sous-processus C doit décoder en **UTF-8** (`encoding="utf-8"`), sinon le cp1252 par défaut produit du mojibake (« cœur » → « cÅ"ur »). Voir `executeur.py` (`_executer_cape` décode en utf-8 et normalise `\r\n` → `\n`) et `tuteur_ia.demander_aide`.
- **Pas de fenêtre cmd qui clignote** — l'appli est packagée `--windowed` ; chaque gcc/test ouvrirait une console. Le drapeau `CREATE_NO_WINDOW` est passé partout (`_SANS_FENETRE` dans `executeur.py` / `tuteur_ia.py`, `chemins.SANS_FENETRE`). Vaut `0` hors Windows, sans effet.
- **Auto-localisation des outils** — `executeur.assurer_compilateur_sur_path()` (gcc) et `executeur.assurer_clangd_sur_path()` (clangd) ajoutent `w64devkit\bin` / `clangd\bin` au PATH s'ils sont trouvés à côté de l'appli. Appelées au tout début de `atelier_snake.main()` : gcc et clangd marchent donc **même au double-clic direct sur l'exé**, pas seulement via `lancer.bat`. Ne font rien si l'outil est déjà sur le PATH.
- **Bootstrap `sys.path`** — `atelier_snake.py` fait encore `sys.path.insert(0, str(Path(__file__).resolve().parent))` avec un commentaire sur la distribution « embeddable ». Ce montage embeddable **n'est plus utilisé** (build PyInstaller) ; la ligne reste inoffensive et sert aussi à lancer depuis un Python normal. (*Écart mineur vs le commentaire du code, qui est resté daté.*)
- **Résolution du moteur du tuteur** (`tuteur_ia._moteur_choisi`) — priorité : 1. commande sur mesure (`reglages.commande_ia` ou variable `ATELIER_AI_CMD`, qui l'emporte), le binaire doit exister ; 2. sinon `ATELIER_AI` force un moteur (s'il est sur le PATH) ; 3. sinon auto-détection dans l'ordre `claude`, puis `codex`. Aucun moteur → l'atelier marche sans tuteur. `reglages.tuteur_actif()` peut le couper côté enseignant.
- **Choix du parcours** — `atelier_snake._parcours_choisi()` : `--parcours <nom>` / `--parcours=<nom>` d'abord, sinon `reglages.dernier_parcours()` (défaut `be_c`). Le point

d'entrée du bundle `packaging/entree_be_c.py` **force** `--parcours be_c` si aucun n'est déjà passé (l'exé n'a pas d'arguments au double-clic).

5. Build & empaquetage

Script de référence : `packaging/build_windows.ps1` . Commande typique :

```
powershell -ExecutionPolicy Bypass -File packaging\build_windows.ps1 -Zip -SkipInstall
```

Ce qu'il fait (étapes idempotentes) :

1. Trouve **Python 3.12** (`%LOCALAPPDATA%\Programs\Python\Python312\python.exe` , `py -3.12 ...`) ; installe via winget sauf `-SkipInstall` .
2. `pip install pyinstaller pyqt6 markdown` (sauf `-SkipInstall`).
3. Télécharge/extrait **w64devkit** (gcc) à la racine du dépôt s'il manque.
4. Télécharge/extrait **clangd** à la racine, en élaguant ses runtimes sanitizer (~30 Mo).
5. **PyInstaller** depuis `packaging/entree_be_c.py` : `--windowed --name TP-C-perso , --paths <repo> , --add-data "<repo>\contenu\be_c;contenu\be_c"` . → `.build\dist\TP-C-perso\` .
6. Régénère les captures (`outils/captures_doc.py`).
7. Assemble `_bundle\TP-C-perso\` : exe, **w64devkit élagué** (retire g++ , gfortran , gdb , cmake , ninja , vim... ~567→319 Mo ; garde `lib\ / include\`) , `clangd` , `lancer.bat` , `diagnostic.bat` , `GUIDE.md` → `README.md` , `README.pdf` (`outils/doc_pdf.py`) , captures. Vérifie que **gcc répond après élagage**.
8. Vérifie que l'exé démarre en `QT_QPA_PLATFORM=offscreen` , puis nettoie les résidus de ce démarrage (à la racine **et** dans `_internal\`).
9. `-Zip` → `_bundle\TP-C-perso.zip` (via `tar.exe` natif), prêt pour une Release.

Le bundle n'embarque **que le parcours be_c** (via `--add-data`). `lancer_demo.bat` n'est volontairement **pas** copié.

Autres artefacts d'empaquetage / lanceurs :

- `packaging/build_linux.sh` — équivalent Linux.
- `packaging/lancer.bat` , `lancer.sh` , `diagnostic.bat` , `lancer_demo.bat` (interne) ; `Atelier.bat` à la racine (détecte Python 3.12 / `py` et lance depuis le source).
- `outils/` : `captures_doc.py` (captures Qt sans écran), `doc_pdf.py` (Markdown→PDF via Chromium headless), `demo_smoketest.py` (smoke-tests du correcteur, mode démo), `stress_correcteur.py` (robustesse de `porte_programme` , sans IA), `jailbreak_tuteur.py` (stress du garde-fou anti-solution).

Flux de publication en **Release GitHub** : voir `RECONSTRUCTION.md` .

6. Tests

Depuis la racine du dépôt :

```
python -m unittest discover -s tests
```

Prérequis Windows, tous nécessaires :

- `PYTHONUTF8=1` — sinon les assertions contenant des accents échouent (encodage local).
- `w64devkit\bin` **sur le PATH** — pour que gcc soit trouvé (sans lui, la moitié des tests échouent avec « gcc introuvable », ce qui n'est **pas** l'échec « par conception » ci-dessous).
- Utiliser le vrai interpréteur :
`C:\Users\sami\AppData\Local\Programs\Python\Python312\python.exe`. Le `python` nu du PATH est le **stub Microsoft Store** (`...\WindowsApps\python.exe`) qui ne fait qu'ouvrir le Store — vérifié.

Résultat vérifié sur cette machine (Python 3.12, gcc 16.1.0, `PYTHONUTF8=1`, `w64devkit\bin` sur le PATH) :

```
Ran 216 tests ... FAILED (failures=8, skipped=1)
```

Les **8 échecs sont attendus sous Windows** : ils portent tous sur le contenu SDL3 (`jalon / projet`), qui exige SDL3 + pkg-config + les chemins de dev Linux (`chemins.SNAKE_ROOT`, `BUILD_COPY`, `SDL_INCLUDES`) absents ici. Concernés : `test_executeur.TestJalon / TestApercu`, `test_executeur_projet.*`, `test_parcours_projet.*`. Le parcours livré **be_c passe** (ex. `test_parcours_tp` OK).

Commandes de smoke rapides (sans écran) :

```
python atelier_snake.py --smoketest    # construit la fenêtre sans l'afficher
python atelier_snake.py --selftest     # vérifie des portes de référence
python atelier_snake.py --releve       # écrit releve.txt et l'affiche
```

7. Branches & livraison

Constaté (voir `HANDOFF-audit-2026-07-21.md`) :

- Branche de travail courante : `livraison` (équivalente à `livraison-windows`).
- Elle a divergé de `origin/version-projet` (la lignée Linux ; `origin/HEAD` → `origin/version-projet`).
- Le livrable est poussé sur `origin/livraison` ; `origin/livraison-windows` existe aussi.